

24 | 生产稳定的秘密武器：如何实施蓝绿发布？

王炜 · 云原生架构与GitOps实战



你好，我是王炜。今天是我们 GitOps 高级发布策略的第一课。

在之前的课程里，我们通过构建 GitOps 工作流实现了自动发布。不过，我们并没有专门去关注新老版本在做更新时是如何切换流量的，这是因为 Kubernetes 的 Service 和 Pod 滚动更新机制自动帮助我们完成了这部分的工作。

在实际的生产环境中，为了提高发布的可靠性，我们通常需要借助发布策略来更加精细地控制流量切换。在几种发布策略中，蓝绿发布是较为简单且容易理解的一种，所以，我将从它开始来介绍如何在 GitOps 工作流中实施蓝绿发布。

那么，什么是蓝绿发布呢？

蓝绿发布核心思想是：**为应用提供两套环境，并且可以很方便地对它们进行流量切换。**

在一次实际发布过程中，新版本的应用将以“绿色”环境部署到生产环境中，但在流量切换之前它并不接收外部流量。当我们完成“绿色”环境的测试之后，可以通过流量切换的方式让“绿色”环境接收外部请求，而旧的“蓝色”环境并不会立即销毁，而是作为灾备来使用。一旦发布过程产生故障，我们就可以将流量立即切换到旧的“蓝色”环境下。

这种部署方式比较适合那些存在兼容问题，或者因为状态原因导致不能很好地使用 Kubernetes 滚动更新的应用。还有的项目希望在更新时部署一个新的版本，同时控制流量切换过程；或者是在发布出现问题时快速回滚。蓝绿发布也是不错的选择。

在这节课，首先我会通过一个例子来说明如何通过手动的方式来实施蓝绿发布。然后，我会结合 Argo Rollout 这款工具进一步向你介绍如何自动化蓝绿发布过程。

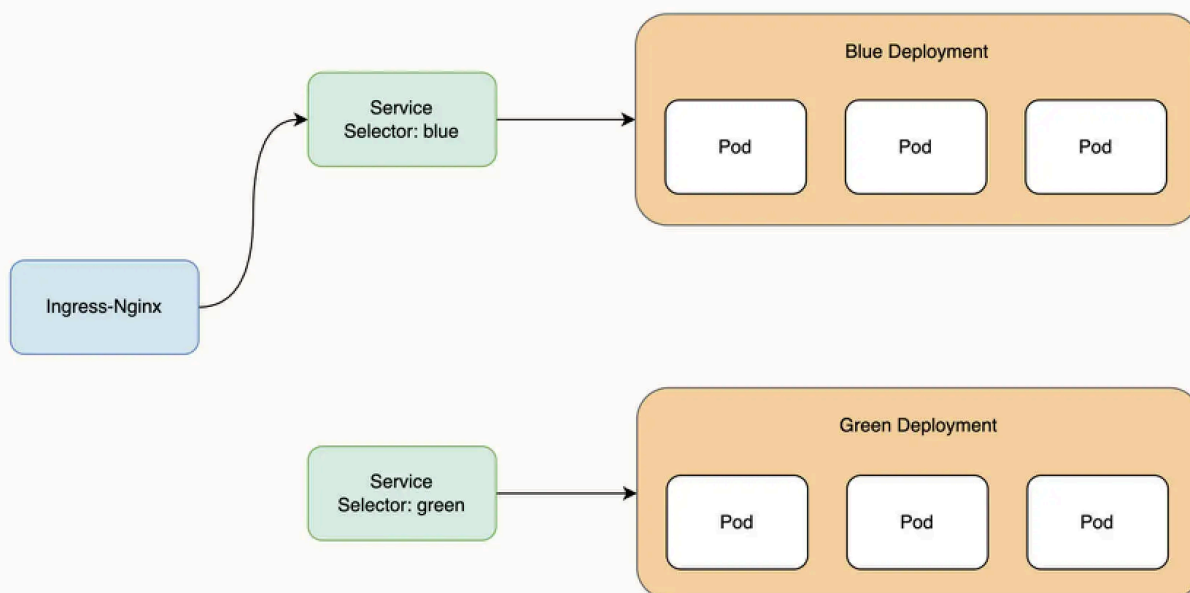
在开始之前，你需要具备以下前提条件。

按照第一章 [🔗 第 2 讲](#) 的内容在本地配置好 Kind 集群，安装 Ingress-Nginx，并暴露 80 和 443 端口。

配置好 Kubectl，使其能够访问 Kind 集群。

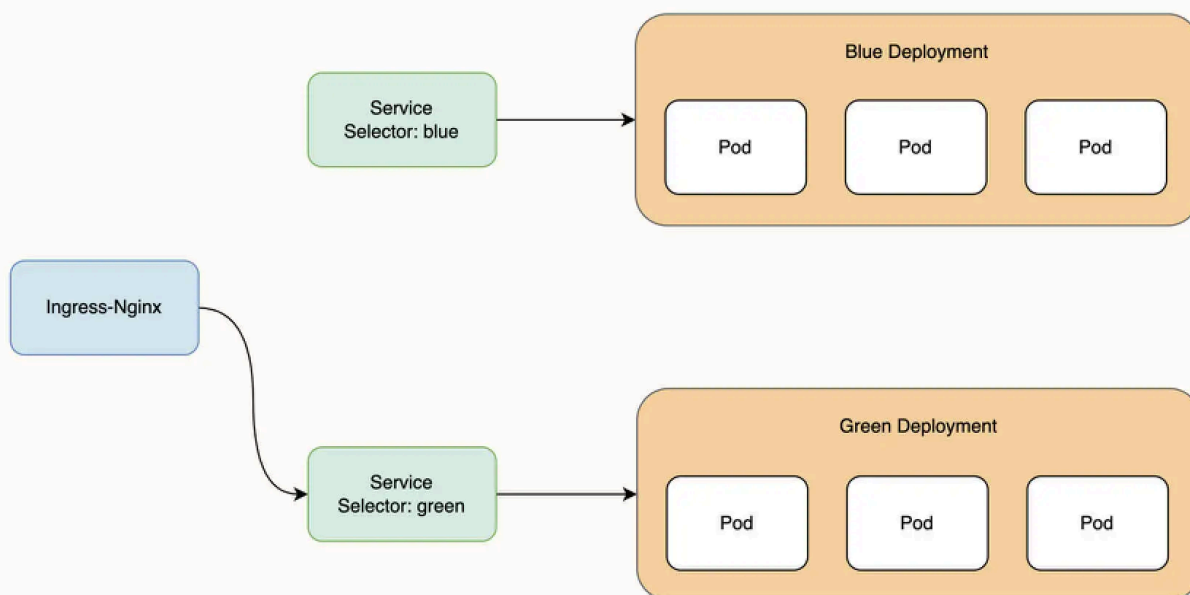
蓝绿发布概述

为了更好地帮助你理解蓝绿发布，在正式进入实战之前，我们先来了解它的整体架构，如下图所示。



在上面这张架构图中，我们对同一个应用部署了两个版本的环境，称之为蓝绿环境，流量通过 Ingress-Nginx 进入到 Service，然后再由它将流量转发至 Pod。在没有切换流量之前，“蓝色”环境负责接收外部请求流量。

需要进行流量切换时，只要调整 Ingress 策略就可以让“绿色”环境接收外部流量，如下图所示。



蓝绿发布实战

接下来，我们进入蓝绿发布实战。我会通过一个例子来说明如何使用 Kubernetes 原生的 Deployment 和 Service 来进行蓝绿发布，实战过程主要包含下面几个步骤。

1. 创建蓝色环境的 Deployment 和 Service。
2. 创建 Ingress 策略，并指向蓝色环境的 Service。
3. 访问蓝色环境。
4. 创建绿色环境的 Deployment 和 Service。
5. 更新 Ingress 策略，并指向绿色环境。
6. 访问绿色环境。

创建蓝色环境

首先，我们需要创建蓝色环境，将下面的内容保存为 blue_deployment.yaml。

```
1 apiVersion: apps/v1
2 kind: Deployment
```

```
3 metadata:
4   name: blue
5   labels:
6     app: blue
7 spec:
8   replicas: 3
9   selector:
10    matchLabels:
11      app: blue
12   template:
13     metadata:
14       labels:
15         app: blue
16     spec:
17       containers:
18       - name: demo
19         image: argoproj/rollouts-demo:blue
20         imagePullPolicy: Always
21         ports:
22         - containerPort: 8080
23 ---
24 apiVersion: v1
25 kind: Service
26 metadata:
27   name: blue-service
28   labels:
29     app: blue
30 spec:
31   ports:
32   - protocol: TCP
33     port: 80
34     targetPort: 8080
35   selector:
36     app: blue
37   type: ClusterIP
```

在上面这段 Manifest 中，我们使用了 `argoproj/rollouts-demo:blue` 镜像创建了蓝色环境的 Deployment 工作负载，并且创建了名为 `blue-service` 的 Service 对象，同时通过 Service 选择器将 Service 和 Pod 进行了关联。

然后，使用 `kubectl apply` 命令将示例应用部署到集群内。

```
1 $ kubectl apply -f blue_deployment.yaml
2 deployment.apps/blue created
3 service/blue-service created
```

部署完成后，等待工作负载 Ready。

```
1 $ kubectl wait pods -l app=blue --for condition=Ready --timeout=90s
2 pod/blue-79c9fb755d-9b6xx condition met
```

 复制代码

当看到上面的输出后，代表绿色环境已经准备好了。

接下来，我们再创建蓝色环境的 Ingress 策略。将下面的内容保存为 blue_ingress.yaml 文件。

```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4   name: demo-ingress
5 spec:
6   rules:
7     - host: "bluegreen.demo"
8     http:
9       paths:
10        - pathType: Prefix
11          path: "/"
12          backend:
13            service:
14              name: blue-service
15              port:
16                number: 80
```

 复制代码

然后，通过 kubectl apply 将其应用到集群内。

 复制代码

```
1 $ kubectl apply -f blue_ingress.yaml
2 ingress.networking.k8s.io/demo-ingress created
```

在上面创建的 Ingress 策略中，我们指定了 bluegreen.info 作为访问域名。所以，在访问蓝色环境之前，你需要先在本地配置 Hosts 才能访问。

```
1 127.0.0.1 bluegreen.demo
```

 复制代码

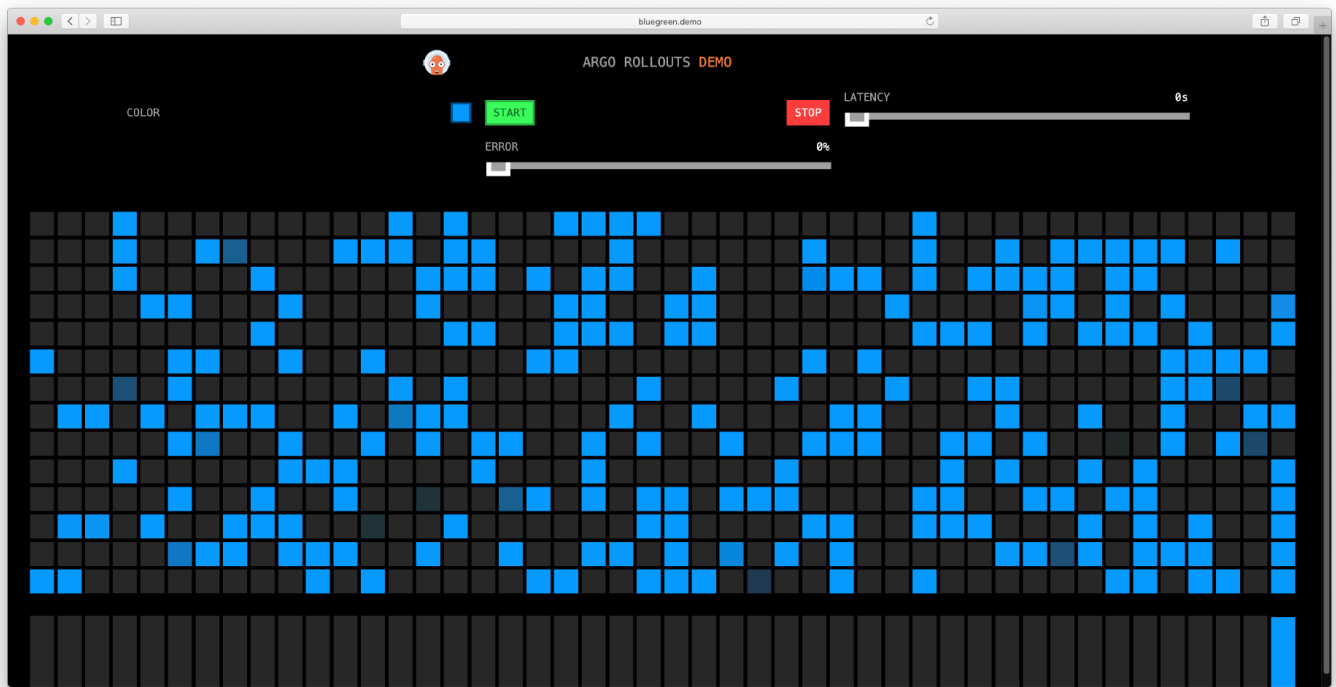
如果你用的是 Linux 或者 MacOS 系统，请将上面的内容添加到 /etc/hosts 文件。

如果你用的是 Windows 系统，需要将上面的内容添加到 C:\Windows\System32\Drivers\etc\hosts 文件。

此外，你还可以使用 [🔗 SwitchHosts](#) 这款开源工具来管理 Hosts 配置。

访问蓝色环境

配置完 Hosts 之后，接下来我们就可以访问蓝色环境了。使用浏览器访问 [🔗 http://bluegreen.demo](http://bluegreen.demo)，你应该能看到如下所示的页面。



在这个页面里，浏览器每秒钟会向后端发出 50 个请求，蓝色的方块代表后端返回接口的内容为 blue，对应 blue 版本的镜像，代表蓝色环境。

部署绿色环境

现在，假设我们需要发布新版本，也就是部署绿色环境。你可以将下面的内容保存为 green_deployment.yaml。

 复制代码

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: green
5   labels:
6     app: green
7 spec:
8   replicas: 3
9   selector:
10    matchLabels:
11      app: green
12   template:
13     metadata:
14       labels:
15         app: green
```

```
16     spec:
17       containers:
18       - name: demo
19         image: argoproj/rollouts-demo:green
20         imagePullPolicy: Always
21         ports:
22         - containerPort: 8080
23 ---
24 apiVersion: v1
25 kind: Service
26 metadata:
27   name: green-service
28   labels:
29     app: green
30 spec:
31   ports:
32   - protocol: TCP
33     port: 80
34     targetPort: 8080
35   selector:
36     app: green
37   type: ClusterIP
```

在这段 Manifest 中，我们用 argoproj/rollouts-demo:green 镜像创建绿色环境的 Deployment，并且创建了名为 green-service 的 Service 对象。

接下来，使用 kubectl apply 来将它应用到集群内。

 复制代码

```
1 $ kubectl apply -f green_deployment.yaml
2 deployment.apps/green created
3 service/green-service created
```

部署完成后，等待工作负载 Ready。

 复制代码

```
1 $ kubectl wait pods -l app=green --for condition=Ready --timeout=90s
2 pod/green-79c9fb755d-9b6xx condition met
```

当看到上面的输出后，代表绿色环境已经准备好了。

切换到绿色环境

现在，当绿色环境已经准备好接收外部流量时，我们就可以通过调整 Ingress 策略来切换流量了。将下面的内容保存为 `green_ingress.yaml`。

 复制代码

```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4   name: demo-ingress
5 spec:
6   rules:
7   - host: "bluegreen.demo"
8     http:
9       paths:
10      - pathType: Prefix
11        path: "/"
12      backend:
13        service:
14          name: green-service
15          port:
16            number: 80
```

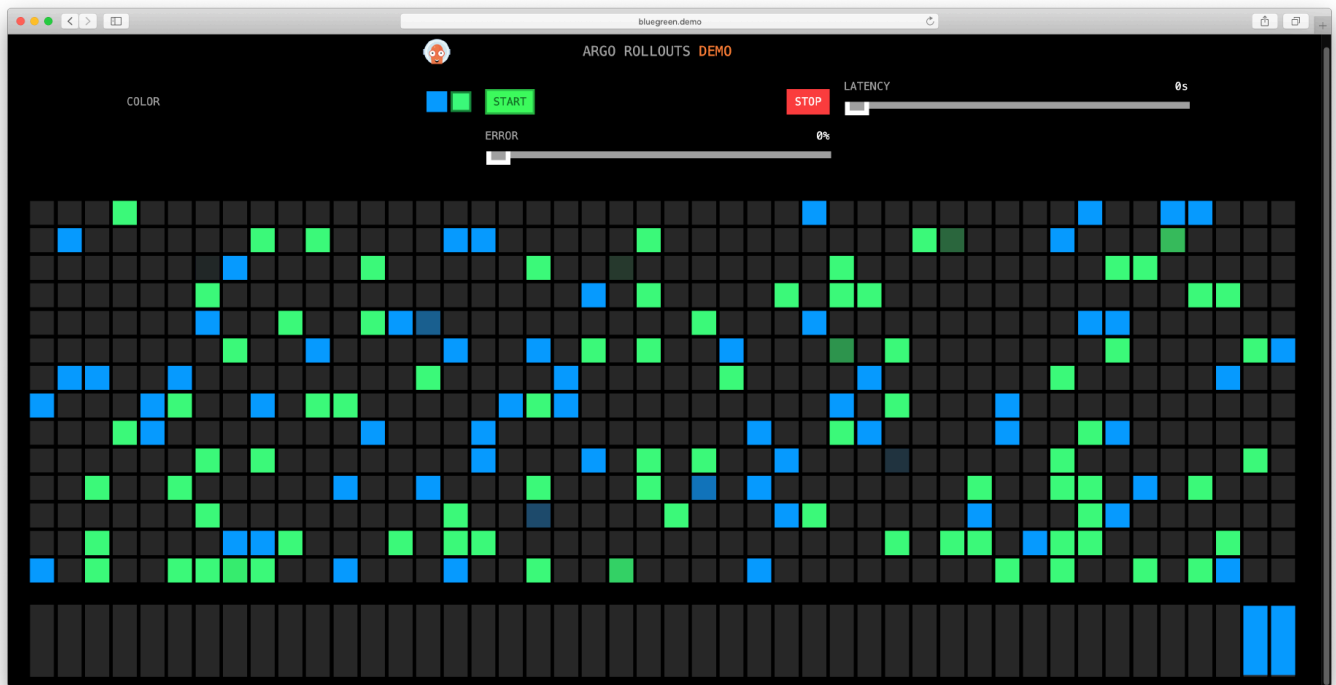
在上面这段 Ingress Manifest 中，我们将 `backend.service` 字段由原来的 `blue-service` 修改为了 `green-service`，这表示将 Ingress 接收到的外部请求转发到绿色环境的 Service 中，以此达到流量切换的目的。

现在，将这段 Ingress 策略应用到集群内。

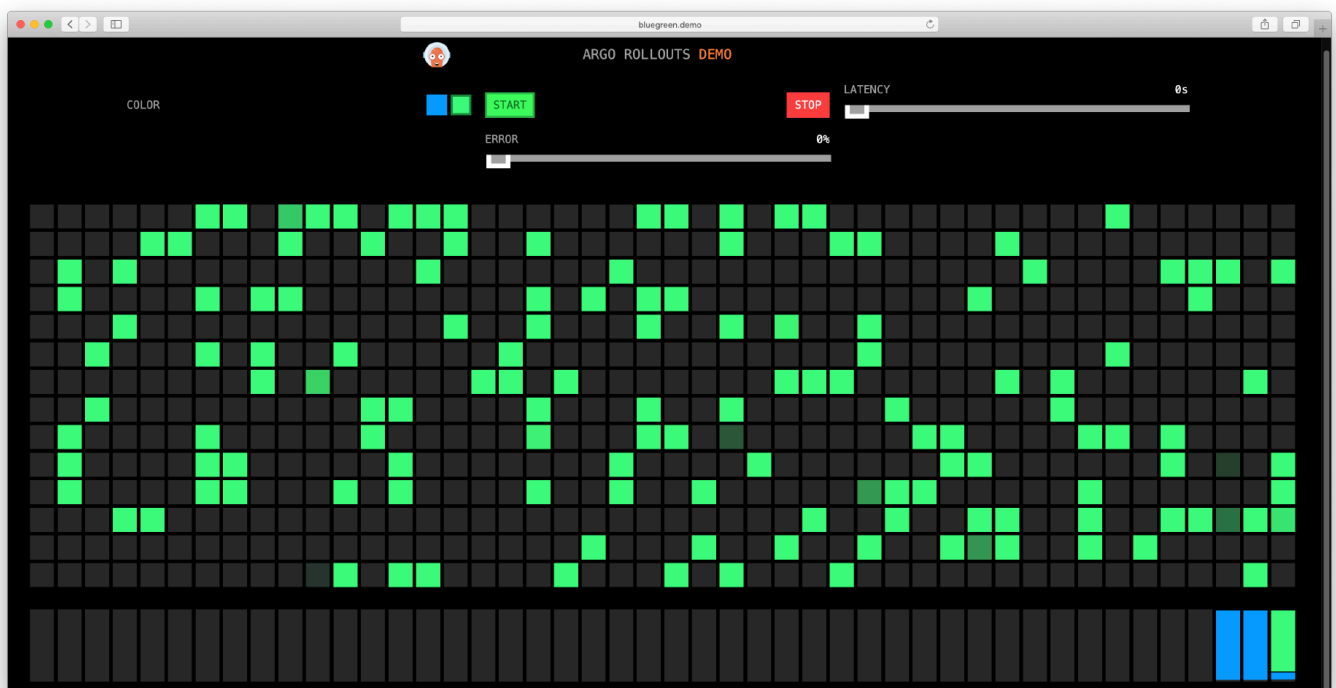
 复制代码

```
1 $ kubectl apply -f green_ingress.yaml
2 ingress.networking.k8s.io/demo-ingress configured
```

重新返回浏览器，你将会看到请求将逐渐从蓝色切换到绿色，如下图所示。



过几秒钟后，请求已经完全变为绿色，这表示流量已经完全从蓝色环境切换到了绿色环境。



到这里，蓝绿发布就已经完成了。

蓝绿发布自动化

到这里，我们都是通过创建 Kubernetes 原生对象并修改 Ingress 策略的方式来完成蓝绿发布的。这存在一些缺点，首先，在更新过程中，我们一般只关注镜像版本的变化，而不会去操作 Ingress 策略；其次，这种方式不利于将蓝绿发布和 GitOps 流水线进行整合。

接下来，我们来看看如何通过 Argo Rollout 工具来自动化蓝绿发布的过程。

安装 Argo Rollout

Argo Rollout 是一款专门提供 Kubernetes 高级部署能力的自动化工具，它可以独立运行，同时也可以和 ArgoCD 协同在 GitOps 流水线中来使用。

在使用之前，我们需要先安装它，你可以通过下面的命令进行安装。

 复制代码

```
1 $ kubectl create namespace argo-rollouts # 创建命名空间
2 $ kubectl apply -n argo-rollouts -f https://ghproxy.com/https://github.com/argopr
```

安装完成后，等待 Argo Rollout 工作负载就绪。

 复制代码

```
1 $ kubectl wait --for=condition=Ready pods --all -n argo-rollouts --timeout=300s
2 pod/argo-rollouts-7f75b9fb76-wh4l5 condition met
```

当看到上面的这段输出后，说明 Argo Rollout 已经准备完成了。

创建 Rollout 对象

和手动实施蓝绿发布的过程不同的是，为了实现自动化，Argo Rollout 采用了自定义资源（CRD）的方式来管理工作负载。如果你暂时还不理解 CRD 也没关系，你只需要知道它是一种扩展 Kubernetes 对象的方式就可以了。

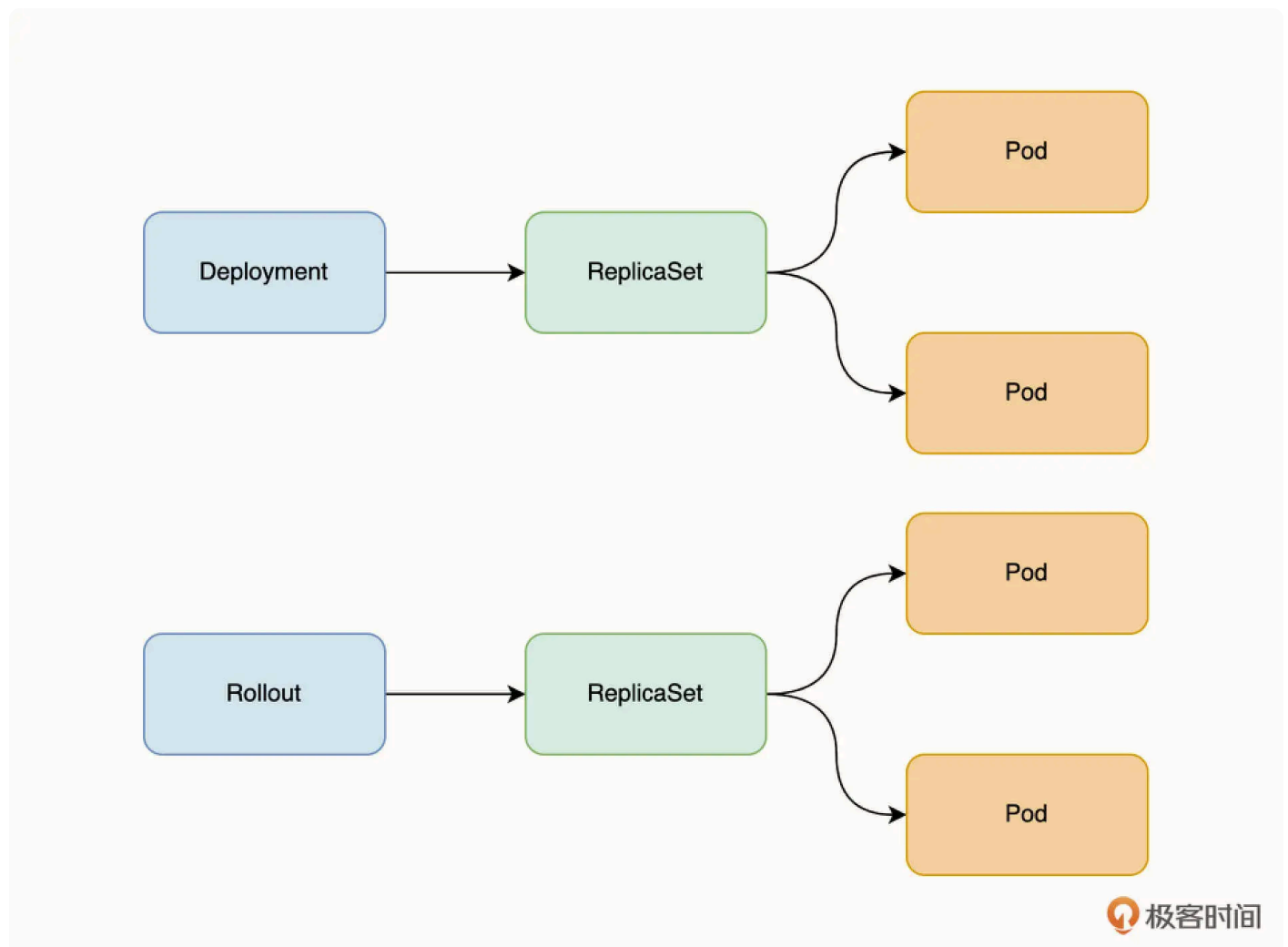
首先，我们需要先创建 Rollout 对象。将下面的内容保存为 blue-green-service.yaml 文件。

```
1 apiVersion: argoproj.io/v1alpha1
2 kind: Rollout
3 metadata:
4   name: bluegreen-demo
5   labels:
6     app: bluegreen-demo
7 spec:
8   replicas: 3
9   revisionHistoryLimit: 1
10  selector:
11    matchLabels:
12      app: bluegreen-demo
13  template:
14    metadata:
15      labels:
16        app: bluegreen-demo
17    spec:
18      containers:
19      - name: bluegreen-demo
20        image: argoproj/rollouts-demo:blue
21        imagePullPolicy: Always
22        ports:
23        - name: http
24          containerPort: 8080
25          protocol: TCP
26        resources:
27          requests:
28            memory: 32Mi
29            cpu: 5m
30  strategy:
31    blueGreen:
32      autoPromotionEnabled: true
33      activeService: bluegreen-demo
```

如果你仔细观察，会发现在这个 Rollout 对象中，它大部分的字段定义和 Kubernetes 原生的 Deployment 工作负载并没有太大的区别，只是将 apiVersion 从 apps/v1 修改为了 argoproj.io/v1alpha1，同时将 kind 字段从 Deployment 修改为了 Rollout，并且增加了 strategy 字段。在容器配置方面，Rollout 对象同样也使用了 argoproj/rollouts-demo:blue 来创建蓝色环境。

需要留意的是，strategy 字段是用来定义部署策略的。其中，autoPromotionEnabled 字段表示自动实施蓝绿发布，activeService 用来关联蓝绿发布的 Service，也就是我们在后续要创建的 Service 名称。

总结来说，当我们将这段 Rollout 对象应用到集群内之后，Argo Rollout 首先会创建 Kubernetes 原生对象 ReplicaSet，然后，ReplicaSet 会创建对应的 Pod。为了帮助你理解，你可以将它与之前手动实施蓝绿发布过程中创建的 Deployment 工作负载进行对比，如下图所示。



从上面这张图我们可以看出，它们的最核心的区别在于 ReplicaSet 是由谁管理的。很显然，在这个例子中，Rollout 对象管理 ReplicaSet 对象，进而达到了管理 Pod 的目的。

在理解了它们的关系之后，接下来我们创建 Rollout 对象。和普通资源一样，你可以通过 `kubectl apply` 来创建。

[📄 复制代码](#)

```
1 $ kubectl apply -f blue-green-rollout.yaml
2 rollout.argoproj.io/bluegreen-demo created
```

创建 Service 和 Ingress

创建好 Rollout 对象之后，我们还需要创建 Service 和 Ingress 策略，这和之前手动实施蓝绿发布的过程是一致的。

首先，创建 Service。将下面的内容保存为 blue-green-service.yaml 文件。

[📄 复制代码](#)

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: bluegreen-demo
5   labels:
6     app: bluegreen-demo
7 spec:
8   ports:
9     - port: 80
10     targetPort: http
11     protocol: TCP
12     name: http
13   selector:
14     app: bluegreen-demo
```

然后，将它应用到集群内。

[📄 复制代码](#)

```
1 $ kubectl apply -f blue-green-service.yaml
2 service/bluegreen-demo created
```

最后，创建 Ingress 对象。将下面的内容保存为 blue-green-ingress.yaml 文件。

 复制代码

```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4   name: bluegreen-demo
5 spec:
6   rules:
7   - host: "bluegreen.auto"
8     http:
9       paths:
10      - pathType: Prefix
11        path: "/"
12      backend:
13        service:
14          name: bluegreen-demo
15          port:
16            number: 80
```

和之前创建的 Ingress 对象不同的是，这里我们使用了 bluegreen.auto 域名，以便和之前创建的 Ingress 域名区分开。

然后，使用 kubectl apply 命令将它应用到集群内。

 复制代码

```
1 $ kubectl apply -f blue-green-ingress.yaml
2 ingress.networking.k8s.io/bluegreen-demo created
```

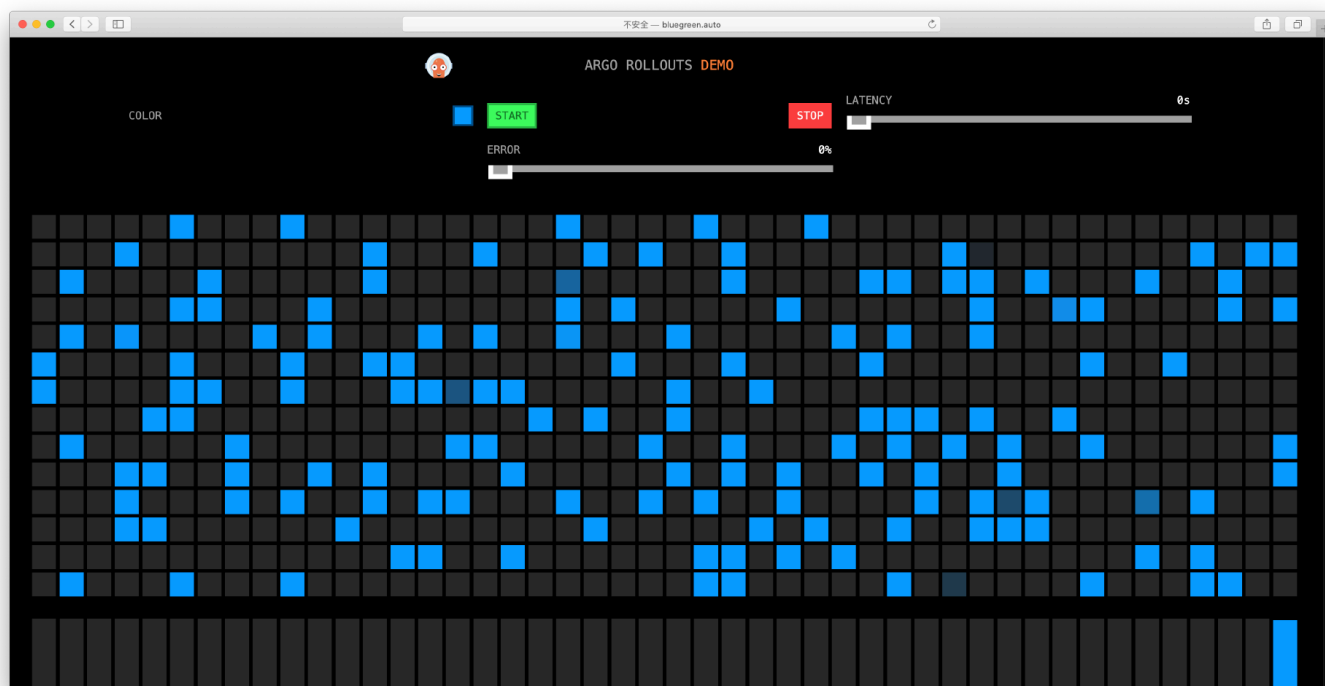
同样地，为了能够访问 bluegreen.auto 域名，你还需要添加 Hosts 策略。

 复制代码

```
1 127.0.0.1 bluegreen.auto
```

访问蓝色环境

配置完 Hosts 之后，接下来我们就可以访问由 Argo Rollout 创建的蓝色环境了。使用浏览器访问 <http://bluegreen.auto>，你应该能看到和手动实施蓝绿发布一样的页面。



蓝绿发布自动化

现在，假设我们需要更新到绿色环境，在 Argo Rollout 的帮助下，你只需要修改 Rollout 对象中的镜像版本就可以了，流量切换过程将由 Argo Rollout 自动控制。

要更新到绿色环境，你需要编辑 `blue-green-rollout.yaml` 文件的 `image` 字段，将 `blue` 修改为 `green` 版本。

 复制代码

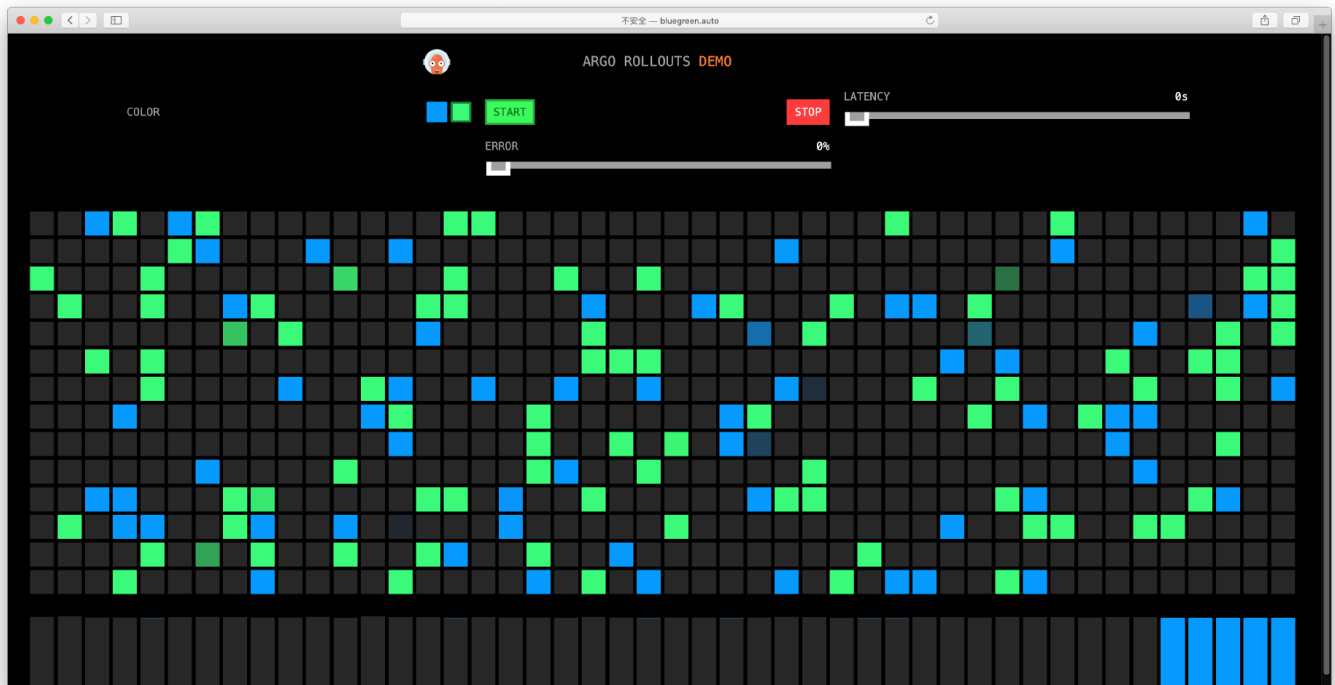
```
1 containers:
2 - name: bluegreen-demo
3   image: argoproj/rollouts-demo:green
```

然后，使用 `kubectl apply` 将这段 Rollout 对象重新应用到集群内。

 复制代码

```
1 $ kubectl apply -f blue-green-rollout.yaml
2 rollout.argoproj.io/bluegreen-demo configured
```

现在，返回到浏览器，等待十几秒后，你应该就能看到请求里开始出现绿色环境了。



几秒钟后，所有请求都变成了绿色方格，这表示蓝绿发布的自动化过程已经完成。

相比较手动的方式，在使用 Argo Rollout 进行蓝绿发布的过程中，我们不再需要手动去切换流量，除了更新镜像版本以外，我们也无需关注其他的 Kubernetes 对象。

访问 Argo Rollout Dashboard

要访问 Argo Rollout Dashboard，首先你需要安装 Argo Rollout 的 kubectl 插件，以 MacOS 为例，你可以通过下面的命令来安装。

```
1 $ brew install argoproj/tap/kubectl-argo-rollouts
```

 复制代码

Linux 或 Windows 系统可以通过直接下载二进制可执行文件的方式来安装，你可以在 [这个链接](#) 下载，并将它加入到 PATH 环境变量中，详细的步骤你可以参考 [这份文档](#)。

插件安装完成后，你可以通过下面的命令来检查安装是否成功。

 复制代码

```
1 $ kubectl argo rollouts version
2 kubectl-argo-rollouts: v1.3.0+93ed7a4
3   BuildDate: 2022-09-19T02:51:42Z
4   GitCommit: 93ed7a497b021051bf6845da90907d67c231e703
5   GitTreeState: clean
6   GoVersion: go1.18.6
7   Compiler: gc
8   Platform: darwin/amd64
```

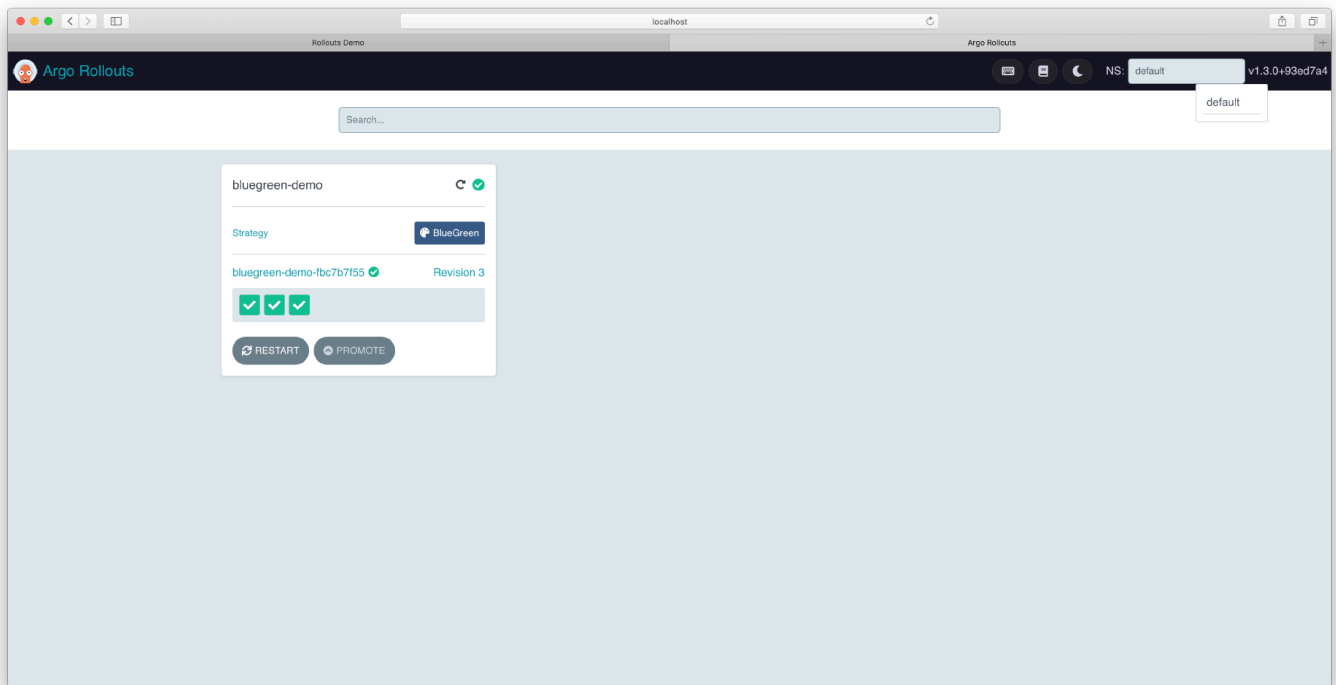
当看到上面的输出结果后，说明插件安装成功了。

接下来，我们可以使用 `kubectl argo rollouts dashboard` 来启用 Dashboard。

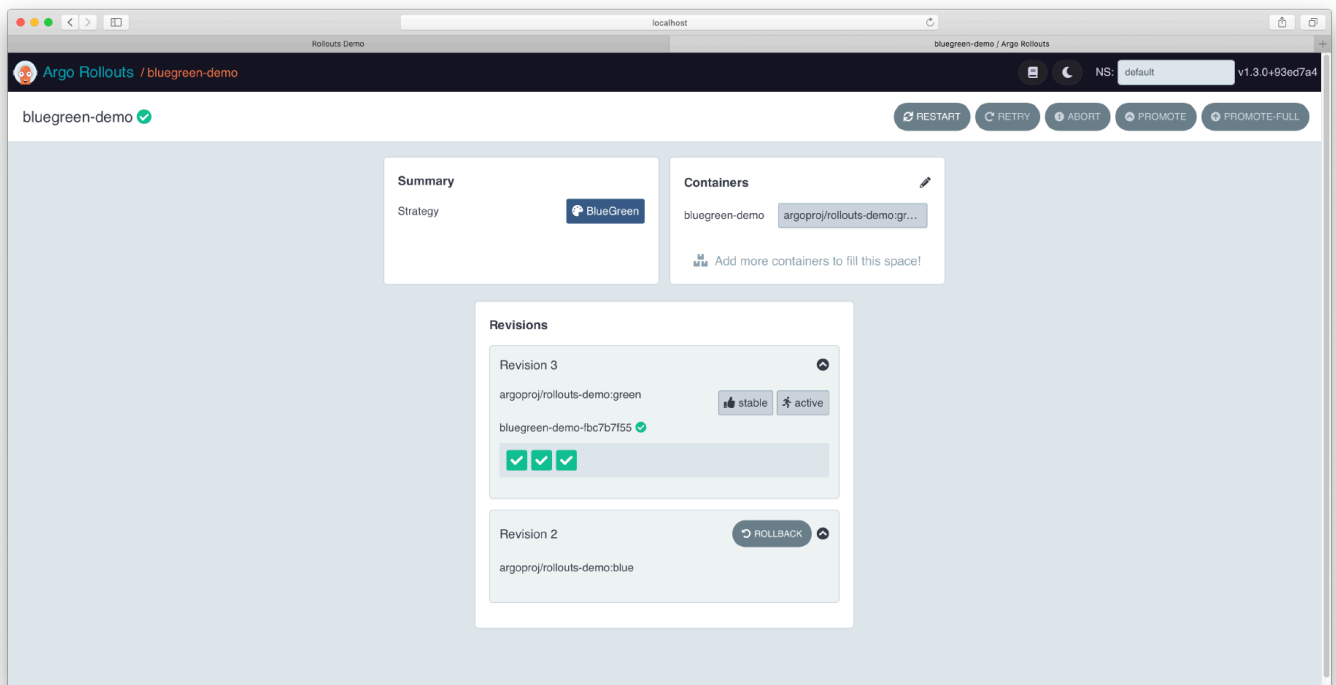
 复制代码

```
1 $ kubectl argo rollouts dashboard
2 INFO[0000] Argo Rollouts Dashboard is now available at http://localhost:3100/roll
```

然后，使用浏览器访问 <http://localhost:3100/rollouts> 打开 Dashboard，如下图所示。



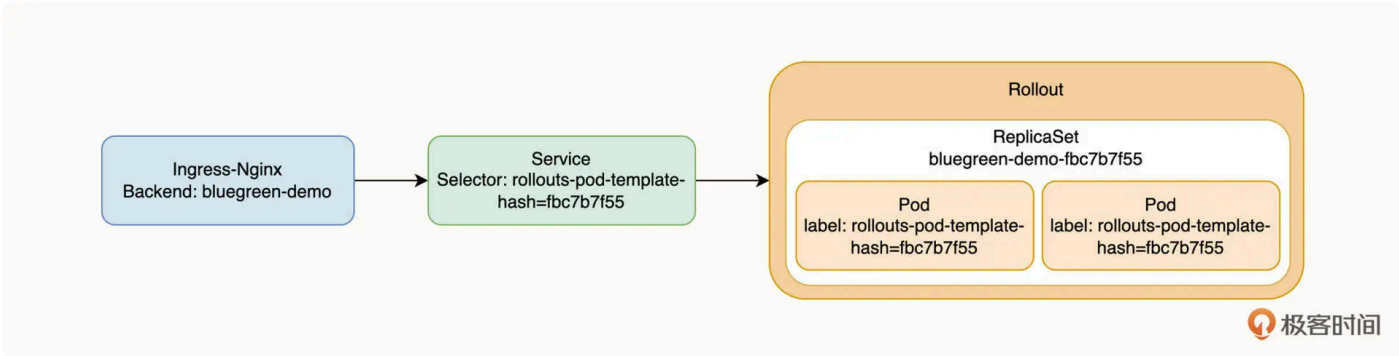
点击进入 Rollout 的详情界面，在这里，你能够以图形化的方式来查看 Rollout 的信息或进行回滚操作。



自动化原理

那么，Argo Rollout 为什么能够帮助我们自动切换流量呢？接下来，我为你详细分析一下它的工作原理。

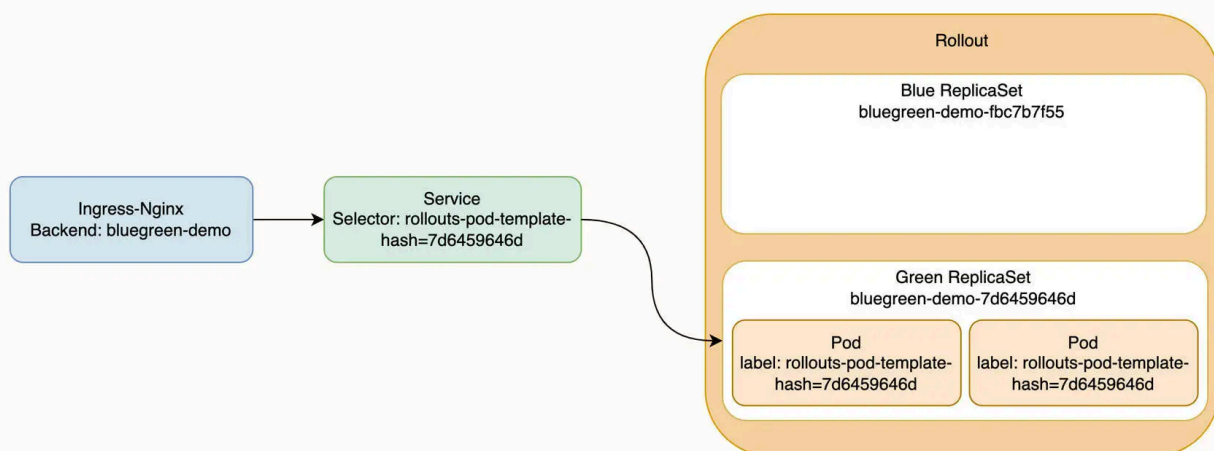
在刚开始创建蓝色环境时，Ingress、Service 和 Rollout 的关系是下图这样。



在这个例子中，当 Rollout 对象创建后，Argo Rollout 将会随之创建 ReplicaSet 对象，名称为 blue-green-fbc7b7f55，这个 ReplicaSet 会在创建 Pod 时额外为 Pod 打上 rollouts-pod-template-hash=fbc7b7f55 的标签，同时为 Service 添加 rollouts-pod-template-hash=fbc7b7f55 选择器，这样，就打通了从 Ingress 到 Pod 的请求链路。

当我们修改 Rollout 对象的镜像版本后，Argo Rollout 将会重新创建一个新的 ReplicaSet 对象，名称为 bluegreen-demo-7d6459646d，新的 ReplicaSet 也会在创建 Pod 时额外为 Pod 打上 rollouts-pod-template-hash=7d6459646d 标签。这时候蓝绿环境的 ReplicaSet 同时存在。

当绿色环境的 Pod 全部就绪之后，Argo Rollout 会将 Service 原来的选择器删除，并添加 rollouts-pod-template-hash=7d6459646d 的选择器，这样就将 Service 指向了绿色环境的 Pod，从而达到了切换流量的目的。同时，Argo Rollout 还会将蓝色环境的 ReplicaSet 副本数缩容为 0，但并不删除它，而是把它作为灾备。如下图所示。



这样，当我们需要重新回滚到蓝色环境时，Argo Rollout 只需调整蓝色环境的 ReplicaSet 副本数并且修改 Service 的选择器，就可以达到快速回滚的目的。

总结

这节课，我首先通过手动的方式带你实践了蓝绿发布的过程。这个过程的核心是部署两套 Deployment 和 Service，同时通过修改 Ingress 策略来实现切换流量。

但手动的方式并不适合与 GitOps 流水线结合使用，所以我们又介绍了通过 Argo Rollout 将蓝绿发布自动化的方法。

要将手动过程切换到自动化过程其实也非常简单，我们只需要安装 Argo Rollout，并修改 Deployment 对象的 apiVersion 和 Kind 字段，然后增加 strategy 字段配置蓝绿发布策略就可以了。

然后，我还为你分析了 Argo Rollout 实现自动化蓝绿发布的原理。和手动修改 Ingress 策略来实现的蓝绿发布不同的是，它主要是通过自动修改 Service 的选择器来对流量进行切换的。这种方式将蓝绿发布的过程变成了更新镜像的操作，极大降低了蓝绿发布的门槛。

最后，你需要注意的是，如果你希望在微服务架构下实施蓝绿发布，那么情况会复杂得多，你需要关注整个微服务链路的蓝绿流量的切换过程，并且在数据库层面也需要考虑对蓝绿发布的支持和适配情况，使数据库在升级过程中能够同时支持蓝绿（新旧）应用。

在下一节课，我会向你介绍在生产环境下更常见的一种发布方式：金丝雀发布。

思考题

最后，给你留两道思考题吧。

1. 在手动实施蓝绿发布的过程中，当流量切换到绿色环境时，如何将蓝色环境的副本数缩容至 0？
2. 在上面的例子中，一旦更新 Rollout 对象的镜像版本，蓝绿发布过程就会自动进行。请你动手试试，如何使用 Rollout 对象的 `autoPromotionEnabled` 参数和 Argo Rollout `kubectl` 插件，实现手动控制蓝绿发布呢？（小提示：你可以参考 [这个链接](#)。）

欢迎你给我留言交流讨论，你也可以把这节课分享给更多的朋友一起阅读。我们下节课见。

AI智能总结

本文深入介绍了如何使用蓝绿发布来稳定生产环境，并重点介绍了在GitOps工作流中实施蓝绿发布的方法。通过示例详细介绍了手动实施蓝绿发布的步骤，包括创建Deployment和Service、Ingress策略、访问蓝色环境、部署绿色环境、切换到绿色环境等。此外，还介绍了如何通过Argo Rollout工具来自动化蓝绿发布的过程。读者可以了解蓝绿发布的核心思想和实际操作步骤，以及如何利用工具来简化和自动化发布过程，从而提高生产环境的稳定性和可靠性。文章还介绍了Argo Rollout的工作原理，帮助读者深入理解自动化发布的实现方式。文章还提出了两个思考题，引发读者思考和讨论。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (7)

最新 精选



争光 Alan

2023-03-14 来自上海

我理解rollout就是给deployment做了增强，增加了一种新版本创建时老版本保留的能力

我的几个疑惑

1. 一般业务会有前段，后端，数据库，微服务更多组件，这样的应用如何进行蓝绿？rollout可

以对多个deployment做蓝绿吗？

2. 新老版本共存的情况下，数据库是同一个，数据库新老版本不兼容如何处理？

作者回复: 这两个问题都非常好。

首先文章演示的蓝绿发布实际上是南北流量的蓝绿，微服务内部的东西流量灰度需要借助 istio 服务网格来实现，当业务内部发起服务间调用时，透传灰度标识，这样才能够实现全链路的灰度。

其次数据库的灰度更复杂一些，一般的实践是多实例或者扩展 istio 协议。



👍 5



zangchao

2023-02-01 来自天津

收获很多，很感谢老师，正好最近在做服务迁移K8s，需要实现服务的灰度发布、蓝绿发布等。想问下老师当前在K8s实现灰度发布有哪些开源库选择，Argo Rollout是否为实现K8s环境下灰度发布的最优选择？之前我们用Istio做过，最近准备放弃Istio了

作者回复: Argo Rollout 是一个不错的选择，比较轻量。此外，Netflix 开源的 Spinnaker 也可以做灰度发布，不过比较重。

如果你的业务有很多个微服务，在不侵入业务的情况下，东西流量（微服务和微服务之间的调用）的灰度还是需要借助 Service Mesh 的。

共 2 条评论 >

👍 2



BertGeek

2023-02-09 来自上海

请问老师，几个问题：

阿里云ack 集群，Argo Rollout实现蓝绿发布

1. 因为deployment 改为了 Rollout，导致ack 的无状态页面无法看到pod 容器，只能在控制台容器组中看到。
2. Argo Rollout 是否可以监测镜像变化，自动发布新版本
3. Argocd + Rollout ，如何与 Kustomize 集成。

作者回复: 1. 正常现象，你可以用 Lens 一类的 UI 工具来查看

2. Rollout 似乎不能支持这种用法

3. 查看这个文档：<https://github.com/argoproj/argo-rollouts/blob/master/docs/features/kustomize.md>



👍 1



林龍

2023-02-01 来自广东

有个疑问，蓝绿发布这里案例是通过修改ingress的Deployment中的service修改对应的应用。自动化构建的话是通过引入Argo Rollout，请问这个自动化构建时修改Deployment的images有什么区别，它也是会通过ReplicaSet创建新的pod，同时也是新的pod创建成功后才会慢慢销毁旧的pod

作者回复: 蓝绿发布在升级过程会保留老的环境，核心思想冗余。



1



Da Vinci

2023-05-08 来自广东

启动了dashboard，但是获取不到rollout，看stdoutoutput，上面报x509: “Argo CD” certificate is not trusted，这是什么原因啊

作者回复: 我没有遇到过，不过你可以尝试看一下这个 issues: <https://github.com/argoproj/argo-cd/issues/6048>



托瓦斯克

2023-10-21 来自广东

要使用 Rollout 对象的 `autoPromotionEnabled` 参数和 Argo Rollout kubectl 插件来实现手动控制蓝绿发布，您可以按照以下步骤进行操作：

1. 确保已安装 Argo Rollout kubectl 插件。您可以参考 Argo Rollout 文档中的安装指南来安装插件：<https://argoproj.github.io/argo-rollouts/installation/>
2. 创建一个 Rollout 对象，并设置 `autoPromotionEnabled` 参数为 `false`，以禁用自动推进（auto-promotion）功能。示例配置如下：

```
```yaml
```

```
apiVersion: argoproj.io/v1alpha1
```

```
kind: Rollout
```

```
metadata:
```

```
 name: my-rollout
```

```
spec:
```

```
strategy:
 blueGreen:
 autoPromotionEnabled: false
其他配置项...
```
```

在上面的配置中，我们将 `autoPromotionEnabled` 设置为 `false`，以禁用自动推进功能。

3. 使用 `kubectl` 命令创建或更新 Rollout 对象：

```
```shell
kubectl apply -f rollout.yaml
```
```

确保将 `rollout.yaml` 替换为您实际的配置文件路径。

4. 使用 Argo Rollout kubectl 插件来手动控制蓝绿发布过程。您可以使用以下命令列出 Rollout 对象的可用操作：

```
```shell
kubectl argo rollouts list
```
```

此命令将显示可用的 Rollout 对象及其状态。

5. 执行蓝绿发布的步骤。您可以使用以下命令来推进蓝绿发布的不同阶段：

```
```shell
kubectl argo rollouts promote my-rollout
```
```

这将推进 Rollout 对象到下一个可用的蓝绿发布阶段。

```
```shell
kubectl argo rollouts abort my-rollout
```
```

这将中止当前的蓝绿发布过程。

```
```shell
kubectl argo rollouts set image my-rollout my-container=my-image:tag
```
```

这将更新 Rollout 对象中的容器镜像版本。

通过使用这些命令，您可以手动控制蓝绿发布的不同阶段和操作，而不依赖于 `autoPromotionEnabled` 参数的自动推进功能。



ghostwritten

2023-02-20 来自北京

1. 什么是蓝绿发布？

蓝绿发布呢？蓝绿发布核心思想是：为应用提供两套环境，并且可以很方便地对它们进行流量切换。在一次实际发布过程中，新版本的应用将以“绿色”环境部署到生产环境中，但在流量切换之前它并不接收外部流量。当我们完成“绿色”环境的测试之后，可以通过流量切换的方式让“绿色”环境接收外部请求，而旧的“蓝色”环境并不会立即销毁，而是作为灾备来使用。一旦发布过程产生故障，我们就可以将流量立即切换到旧的“蓝色”环境下。

2. 手动蓝绿发布

部署两套环境，镜像tag 分别为blue、green，切换操作其实是修改ingress的 service:

```
name: blue-service # 改为green-service
```

这种操作 Ingress 策略不利于将蓝绿发布和 GitOps 流水线进行整合。

3. 蓝绿发布自动化

3.1 创建命名空间

```
kubectl create namespace argo-rollouts
```

3.2 安装 argo-rollouts

```
kubectl apply -n argo-rollouts -f https://ghproxy.com/https://github.com/argoproj/argo-rollouts/releases/latest/download/install.yaml
```

3.3 等待

```
kubectl wait --for=condition=Ready pods --all -n argo-rollouts --timeout=300s
```

3.4 创建 Rollout 对象——blue-green-rollout.yaml

重点理解：strategy 字段是用来定义部署策略的。其中，autoPromotionEnabled 字段表示自动实施蓝绿发布，activeService 用来关联蓝绿发布的 Service，也就是我们在后续要创建的 Service 名称。

3.5 创建 Service 和 Ingress

```
kubectl apply -f blue-green-service.yaml
```

```
kubectl apply -f blue-green-ingress.yaml
```

3.6 切换操作对象是Rollouts，是编辑 blue-green-rollout.yaml 文件的 image 字段，将 blue 修改为 green 版本，然后 kubectl apply -f blue-green-rollout.yaml
流量切换过程将由 Argo Rollout 自动控制。

3.7 访问 Argo Rollout Dashboard

```
$ brew install argoproj/tap/kubectl-argo-rollouts
```

检查版本

```
kubectl argo rollouts version
```

部署 dashboard

```
kubectl argo rollouts dashboard
```

4. 当我们需要重新回滚到蓝色环境时，Argo Rollout 只需调整蓝色环境的 ReplicaSet 副本数并且修改 Service 的选择器，就可以回滚

